

Understanding the Difference Between Helm Charts and Traditional Kubernetes Deployments

Kubernetes is a powerful container orchestration platform, and deploying applications to it can be done in various ways.

The most common methods include:

1. Writing and applying raw Kubernetes manifests (e.g., `deployment.yaml`, `service.yaml`).
2. Automating the deployment using a CI/CD pipeline (e.g., GitHub Actions, Jenkins).
3. Using Helm charts, which is a package manager for Kubernetes.

Each of these methods has its place, and understanding the key differences will help you make the right choice depending on the complexity and needs of your application.

1. Traditional Kubernetes Deployments Using `deployment.yaml`

In its simplest form, a developer or DevOps engineer writes Kubernetes manifests manually for every resource deployments, services, config maps, secrets, etc.

For example, a `deployment.yaml` might look like this:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: myrepo/myapp:v1.0
          ports:
            - containerPort: 80
```

You then apply this with:

```
kubectl apply -f deployment.yaml
```

This works well for simple use cases. But problems arise when:

- You need to deploy across environments (dev, staging, prod) with different configs.
- You want version control or rollback capability.
- You need to manage complex apps with multiple services and dependencies.

2. CI/CD Pipeline Deployment

This builds on top of the first method by automating the application of manifests. You still use raw yaml files, but you integrate them into a pipeline

For example:

- Build Docker image
- Push to registry
- Deploy using kubectl via the pipeline

For example, in GitHub Actions:

```
- name: Deploy to Kubernetes
  run: kubectl apply -f k8s/
```

This is better because it removes manual intervention.

However, you're still left managing the complexities of different environments manually perhaps through separate folders or templates like deployment-dev.yaml, deployment-prod.yaml, and so on.

This can get messy and error-prone as the application grows.

3. Helm Charts: Kubernetes Package Management

Helm brings a whole new approach. It is a **package manager for Kubernetes** that helps you define, install, and upgrade complex Kubernetes applications using reusable, templated charts.

Instead of writing static deployment.yaml files, you create a Helm chart which consists of:

- Chart.yaml: Metadata about the chart
- values.yaml: Default values (can be overridden per environment)
- templates/: Templated Kubernetes manifests using Go templating

Example template file (templates/deployment.yaml):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.app.name }}
spec:
  replicas: {{ .Values.app.replicas }}
  selector:
    matchLabels:
      app: {{ .Values.app.name }}
  template:
    metadata:
      labels:
        app: {{ .Values.app.name }}
    spec:
      containers:
        - name: {{ .Values.app.name }}
          image: "{{ .Values.app.image }}"
          ports:
            - containerPort: {{ .Values.app.port }}
```

Now your values.yaml defines environment-specific inputs:

```
app:
  name: my-app
  replicas: 2
  image: myrepo/myapp:v1.0
  port: 80
```

You can deploy with:

```
helm install my-app ./my-chart
```

Or upgrade with:

```
helm upgrade my-app ./my-chart
```

Real-World Use Case Comparison

Use Case: A microservices-based ecommerce application with 10+ components (product, cart, payment, auth, UI, etc.)

Using deployment.yaml + CI/CD:

- You would have 10+ YAML files per environment (dev, staging, prod).
- Managing replicas, secrets, config maps, or even container versions per environment is manual.
- Rollbacks are tough. You'd need to re-apply an older manifest.
- You might use environment variables in your CI/CD pipeline to substitute values — a fragile solution.

Using Helm:

- You write one chart with templates.
- You create environment-specific values files like values-dev.yaml, values-prod.yaml.
- Helm automatically renders templates and installs them.
- Rollbacks are one command away: `helm rollback my-app 1`
- It's easier to reuse and share across teams.

Why Helm is Better in Complex Scenarios

- **Templating:** Avoid duplication by defining templates that accept dynamic values.
- **Versioning:** Each deployment via Helm is versioned and easy to roll back.
- **Environments:** Clean separation via different values.yaml for each environment.
- **Dependency Management:** Helm allows you to define dependencies between charts (e.g., your app requires Redis or Postgres).
- **Lifecycle Hooks:** Pre-install, post-install, upgrade hooks give fine-grained control.

Implementation Steps :

Scenario: Deploying a Node.js app using Helm

Install Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

Create a new Helm chart

```
helm create nodejs-app
```

Customize templates

Edit templates/deployment.yaml to suit your app logic.

Edit values.yaml with image name, replica count, ports, etc.

Build and push Docker image

```
docker build -t myrepo/nodejs-app:v1.0 .
```

```
docker push myrepo/nodejs-app:v1.0
```

Set your image in values.yaml

```
image:
```

```
  repository: myrepo/nodejs-app
```

```
  tag: v1.0
```

Install your app

```
helm install my-nodejs ./nodejs-app -f values.yaml
```

Upgrade app with a new version

Update image tag in values.yaml

Run:

```
helm upgrade my-nodejs ./nodejs-app -f values.yaml
```

Rollback if needed

```
helm rollback my-nodejs 1
```

Helm is **not a replacement** for CI/CD — it is a **tool that fits into it**. In fact, the best practice is:

- **Use Helm for packaging and templating**
- **Use CI/CD for automation**

Helm solves the complexity and scalability problem in Kubernetes deployments. YAML files and CI/CD alone are not enough when your application and environments grow.

If you want maintainability, reusability, version control, and rollback Helm is the tool of choice.

When to Use What: Raw YAML, CI/CD Pipelines, or Helm?

In Kubernetes, there are multiple ways to deploy applications from using raw YAML files, integrating those into CI/CD pipelines, to using advanced tools like Helm.

Each method has its strengths and trade-offs. Here's a clear and accurate guide on **when to choose which approach**, based on real-world scenarios and engineering needs.

Raw deployment.yaml and Other Manifest Files

Best For:

- Beginners learning Kubernetes fundamentals.
- Simple applications with minimal deployment complexity.
- One-off or internal tools that don't require long-term maintainability.

Advantages:

- Full control over Kubernetes resource definitions.
- Easy to read, debug, and modify directly.
- No external tool dependency.

Limitations:

- Hard to manage multiple environments (dev, staging, prod) without duplication.
- Difficult to scale as the number of services or configurations grow.
- No built-in templating or packaging.

Real-World Example:

A team manually deploys a small internal dashboard using basic YAML files and `kubectl apply` for each cluster.

CI/CD Pipelines with YAML Manifests

Best For:

- Teams building modern DevOps workflows.
- Applications that benefit from automation, version control, and auditability.
- Environments where Git and pipelines are already used for infra and code.

Advantages:

- Automates deployments triggered by code changes.
- Enables environment-based deployments (e.g., using GitHub Actions, Azure DevOps, GitLab CI).
- Can integrate linting, testing, and security checks.

Limitations:

- YAML duplication if you manage many services or environments manually.
- Harder to manage shared configurations or secrets without extra tooling.
- Still lacks packaging, templating, and rollback support natively.

Real-World Example:

A GitHub Actions workflow deploys microservices to Kubernetes by running

`kubectl apply -f` with manually customized YAMLs per environment (like `deployment-dev.yaml`, `deployment-prod.yaml`).

Helm Charts

Best For:

- Production-grade deployments across multiple environments.
- Applications with reusable or complex configuration templates.
- Teams adopting GitOps, ArgoCD, or platform engineering approaches.

Advantages:

- Templating and dynamic values make it easy to customize deployments.
- Supports versioning, rollback, upgrades, and dependency management.
- Reduces YAML repetition by creating reusable charts.
- Integrates smoothly with CI/CD and GitOps tools.

Limitations:

- Learning curve for Helm templating syntax (`{{ }}` and functions).
- Requires thoughtful structure and values file management.
- Handling secrets securely needs additional tooling like SOPS or SealedSecrets.

Real-World Example:

A fintech company uses Helm to deploy 20+ microservices, each with different configurations for dev, staging, and production.

The same chart is reused with different values.yaml files and deployed using ArgoCD for GitOps-style automated reconciliation.

Scenario	Recommended Tool
Learning Kubernetes basics	Raw YAML
Building a small app with minimal config	Raw YAML or basic CI/CD
Automating deployment with pipelines	CI/CD with YAML
Managing multiple environments or services	Helm Charts
Need for rollback, upgrade, templating	Helm Charts
Adopting GitOps with ArgoCD/Flux	Helm Charts

In short:

Use raw YAML to learn. Use CI/CD for automation. Use Helm to scale.

Each tool complements the others and when combined effectively, they can deliver a robust, reliable, and modern Kubernetes deployment strategy.